

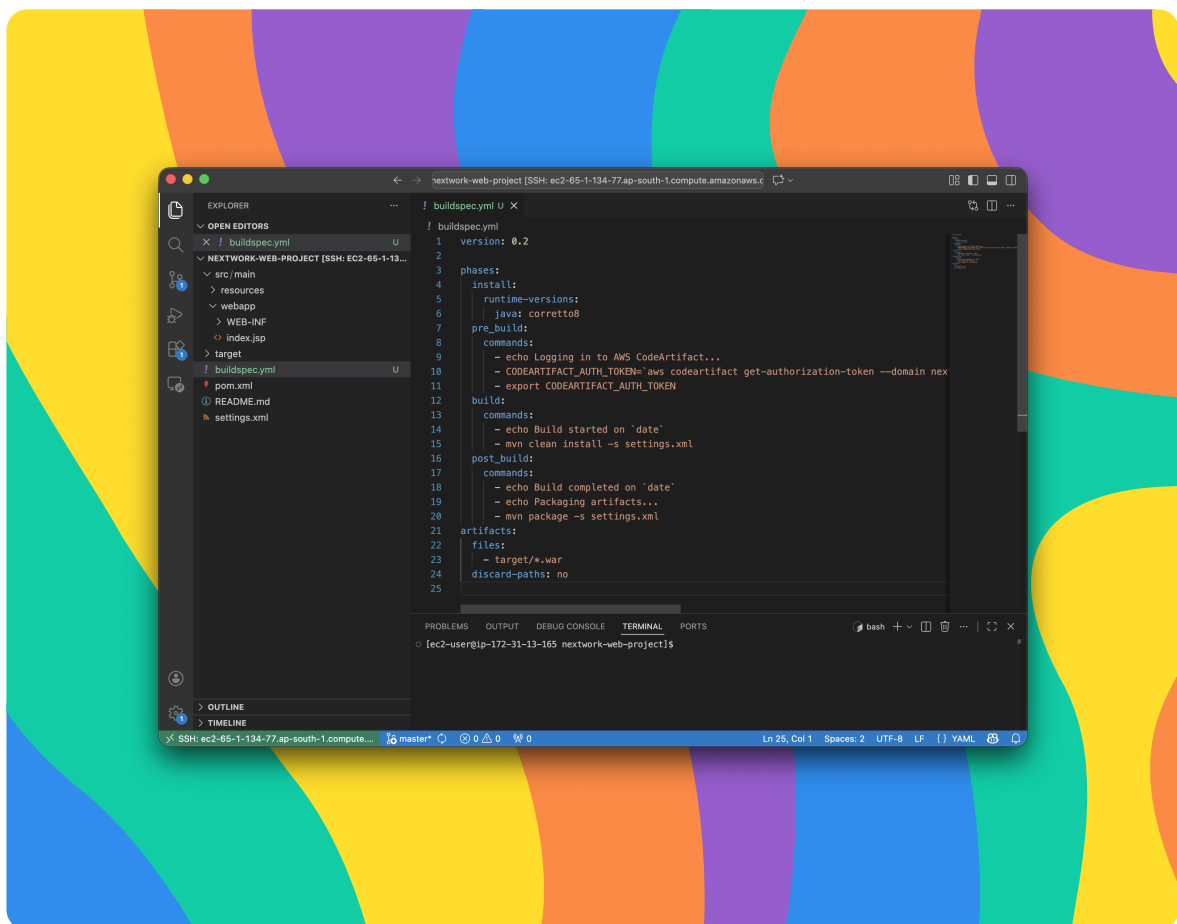


nextwork.org

Continuous Integration with CodeBuild



Dhruva Kashyap





Introducing Today's Project!

The goal is to set up AWS CodeBuild to automatically build the Java web application whenever code changes are pushed to GitHub. This includes creating a CodeBuild project, connecting it to the GitHub repository, and defining the build steps using a 'buildspec.yml'

Key tools and concepts

In this project, I learned how AWS CodeBuild automates the build process by pulling source code from GitHub and executing build steps defined in a buildspec.yml file. I also understood how build artifacts like .war files are packaged and stored in Amazon S3, while CloudWatch Logs helps monitor and troubleshoot build failures through detailed logs. Additionally, I learned how IAM roles and policies enable CodeBuild to securely access AWS CodeArtifact for dependency management and run automated tests as part of a CI workflow.

Project reflection

This project took me approximately 3 hours to complete. The time was mainly spent configuring the CodeBuild environment, integrating it with GitHub, and fixing build failures using buildspec.yml and IAM permissions. It also included verifying build logs, artifacts in S3, and automated testing execution.



Dhruva Kashyap

NextWork Student

nextwork.org

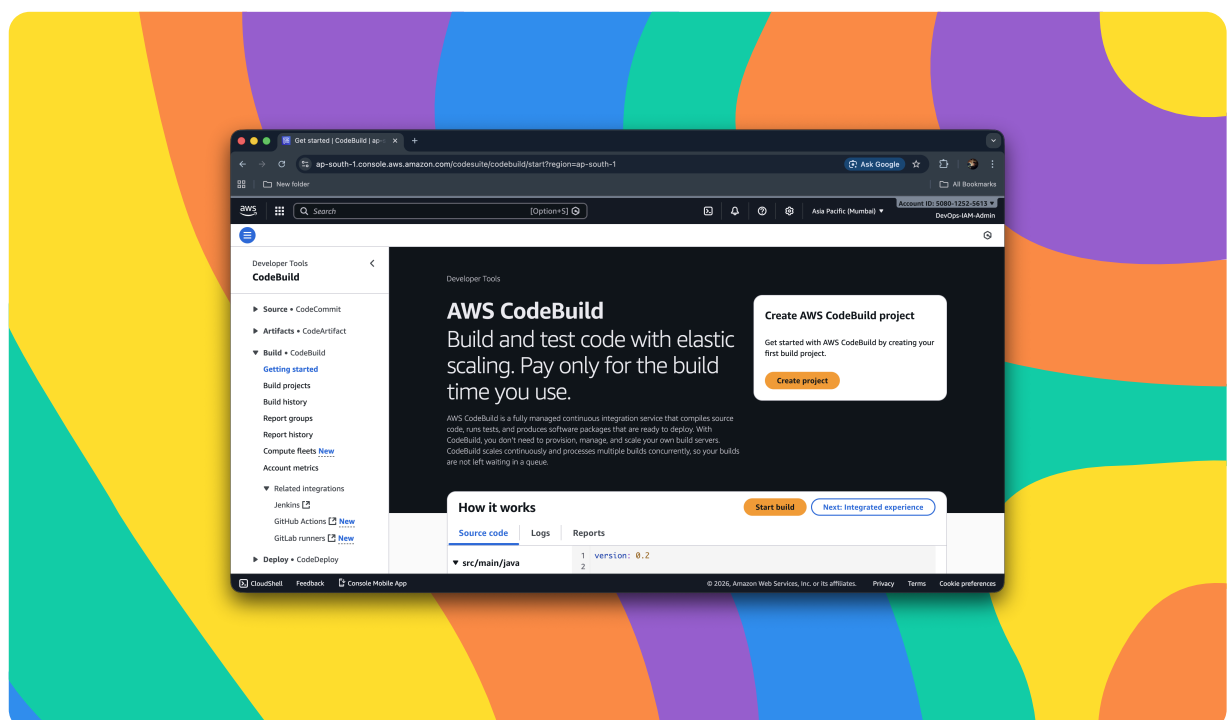
This project is part four of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project. The next project will be done on the upcoming day. It will build on the current setup by extending the CI/CD pipeline with the next AWS service in the workflow. This consistent progress helps strengthen understanding through hands-on implementation.



Setting up a CodeBuild Project

A Continuous Integration (CI) service is a tool that automatically builds and tests your code whenever changes are pushed to the repository. It helps detect errors early by continuously verifying that new updates work correctly with the existing codebase. This ensures the application remains stable and ready for deployment throughout development.

The source of the AWS CodeBuild project refers to where the application's code is stored and fetched from for building. It tells CodeBuild which repository to pull the latest version of the project files before running the build steps. In this setup, GitHub is used as the source so CodeBuild can automatically build the web app from the updated codebase.

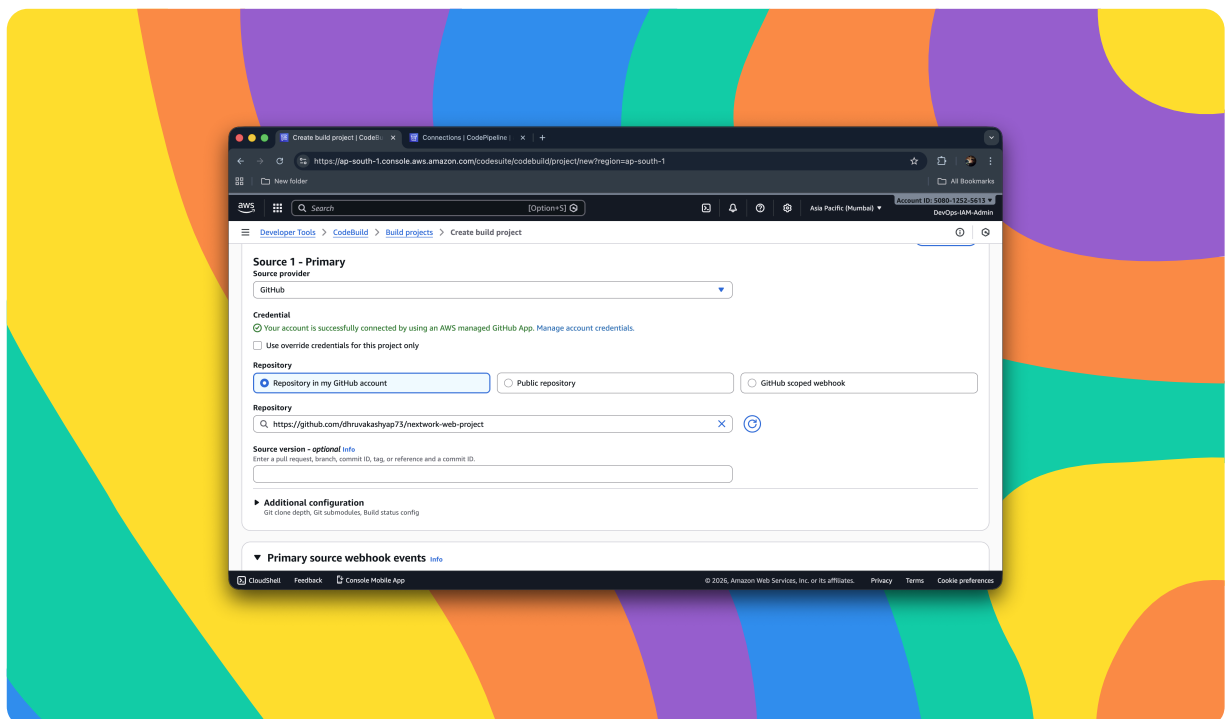




Connecting CodeBuild with GitHub

The GitHub App credential type is recommended because it provides a secure and managed connection without requiring manual token creation or rotation. It reduces the risk of leaked credentials since authentication is handled through AWS-managed integration. This option is also easier to maintain while still allowing AWS CodeBuild to access repositories reliably.

AWS CodeConnections helped by creating a secure, managed integration between the AWS account and the GitHub repository. It handled the authentication through an AWS-managed GitHub App, so no personal tokens or passwords were required. This allowed AWS CodeBuild to access the source code safely for automated builds.





CodeBuild Configurations

Environment

The AWS CodeBuild environment was configured using an on-demand provisioning model with a managed build image to avoid manual tool setup. The build was set to run on Amazon Linux with the Standard runtime and the Corretto 8 image to match the Java requirements of the application. A new service role was also created so CodeBuild can securely access the resources needed during the build process.

Artifacts

The Amazon S3 bucket stores the main build artifact produced after the CodeBuild process completes. This artifact is typically a packaged output of the application, such as a compiled and deployable bundle created from the source code. Storing the artifact in S3 ensures it is centrally available for the next stages of the CI/CD pipeline, such as deployment.

Packaging

Build artifacts are packaged as a ZIP file to bundle all required output files into a single, easy-to-transfer unit. This makes it simpler for other pipeline stages to download, store, and deploy the application consistently. Using ZIP also helps reduce file size through compression and keeps the artifact structure intact across environments.

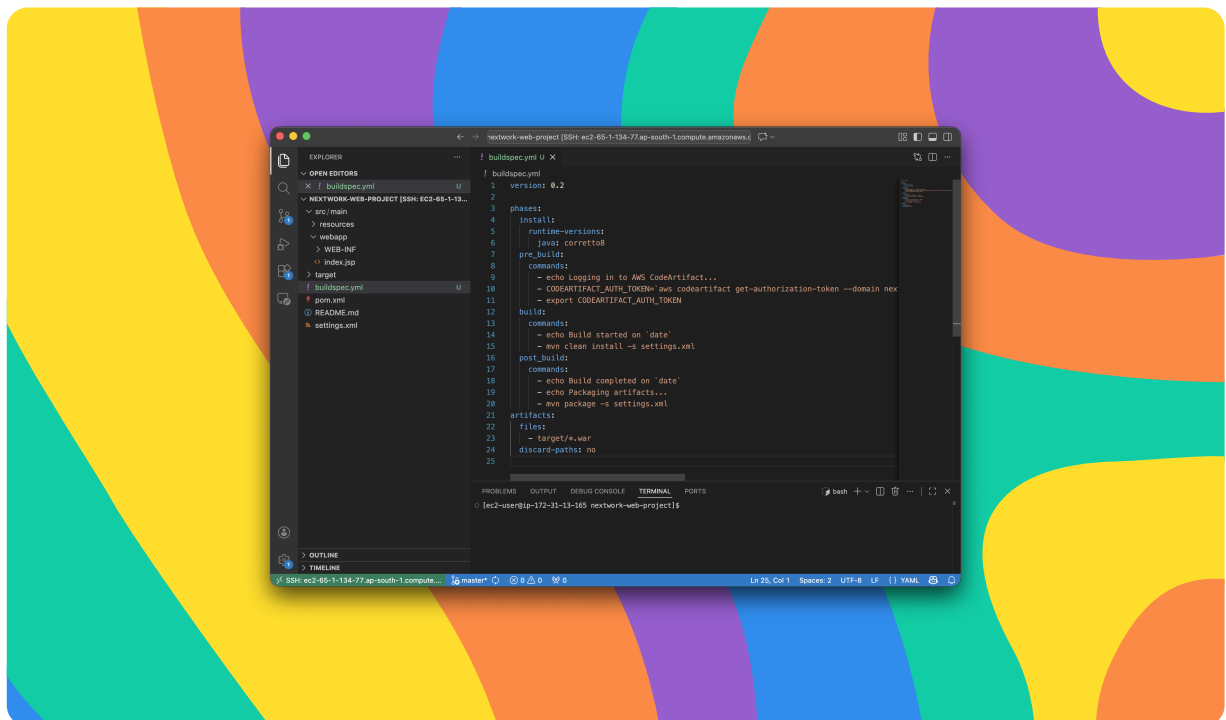
Monitoring

Amazon CloudWatch Logs are used to capture and store build output from AWS CodeBuild in a centralized place. They make it easier to monitor the build process, identify errors, and troubleshoot failures quickly. CloudWatch Logs also provide visibility into build history for auditing and performance tracking.

buildspec.yml

The first build failed because AWS CodeBuild could not find the required buildspec.yml file in the root of the GitHub repository. Without this file, CodeBuild does not have the instructions needed to download dependencies and run the build commands. As a result, the build stopped during the DOWNLOAD_SOURCE phase with a YAML file missing error.

The phases in buildspec.yml define the order of steps that AWS CodeBuild follows during the build process. The install phase prepares the environment and installs required tools or runtimes, while pre_build is used for setup actions such as authentication, environment variables, or dependency configuration. The build phase runs the main tasks like testing and compiling the application, and post_build handles final steps such as packaging artifacts and preparing outputs for deployment.



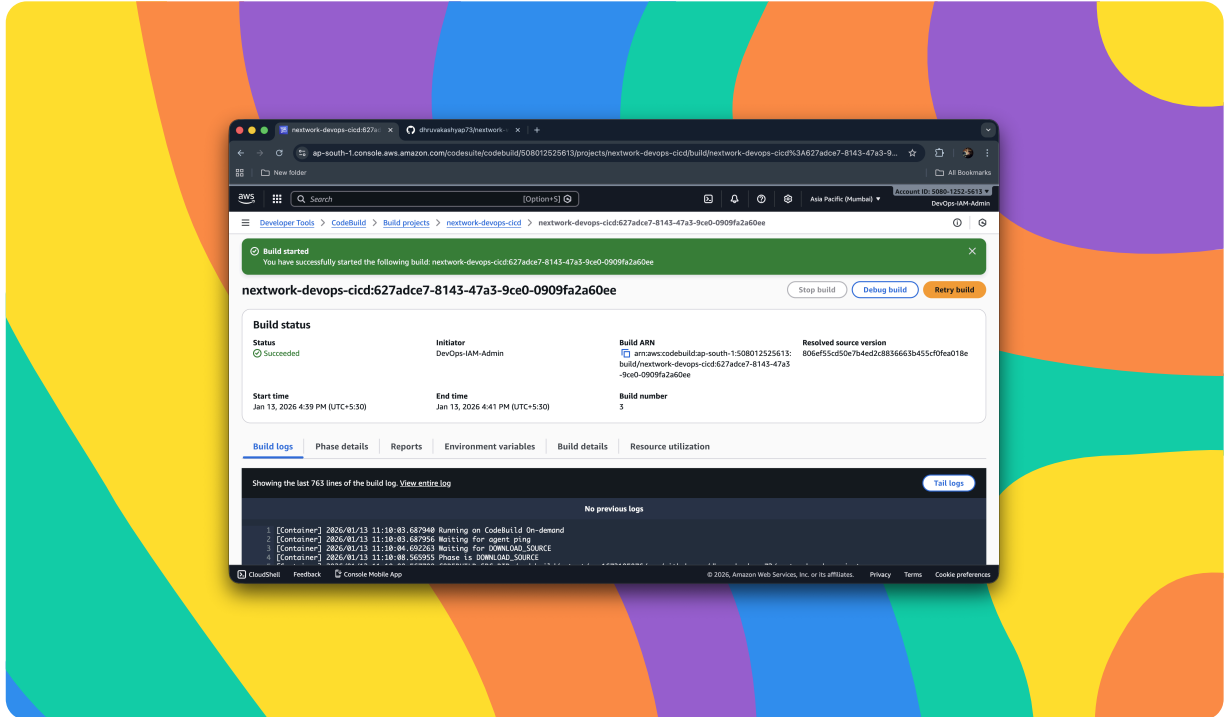


Success!

The second build failed because AWS CodeBuild did not have the required permissions to access AWS CodeArtifact for downloading project dependencies. Even though the source code was downloaded successfully, the build could not authenticate and retrieve packages through the configured settings.xml. This caused the build to fail during the dependency resolution stage due to missing CodeArtifact access.

The error was resolved by attaching the codeartifact-nextwork-consumer-policy to the CodeBuild service role that was created for the project. This update gave AWS CodeBuild the required permissions to authenticate and download dependencies from AWS CodeArtifact. After updating the role permissions, the build was re-run and completed successfully.

After the build completed, the Amazon S3 bucket nextwork-devops-cicd was checked to verify the output. The build process created and uploaded the artifact file nextwork-devops-cicd-artifact.zip into the bucket. Inside this ZIP file, the compiled application package nextwork-web-project.war was generated as the deployable output.



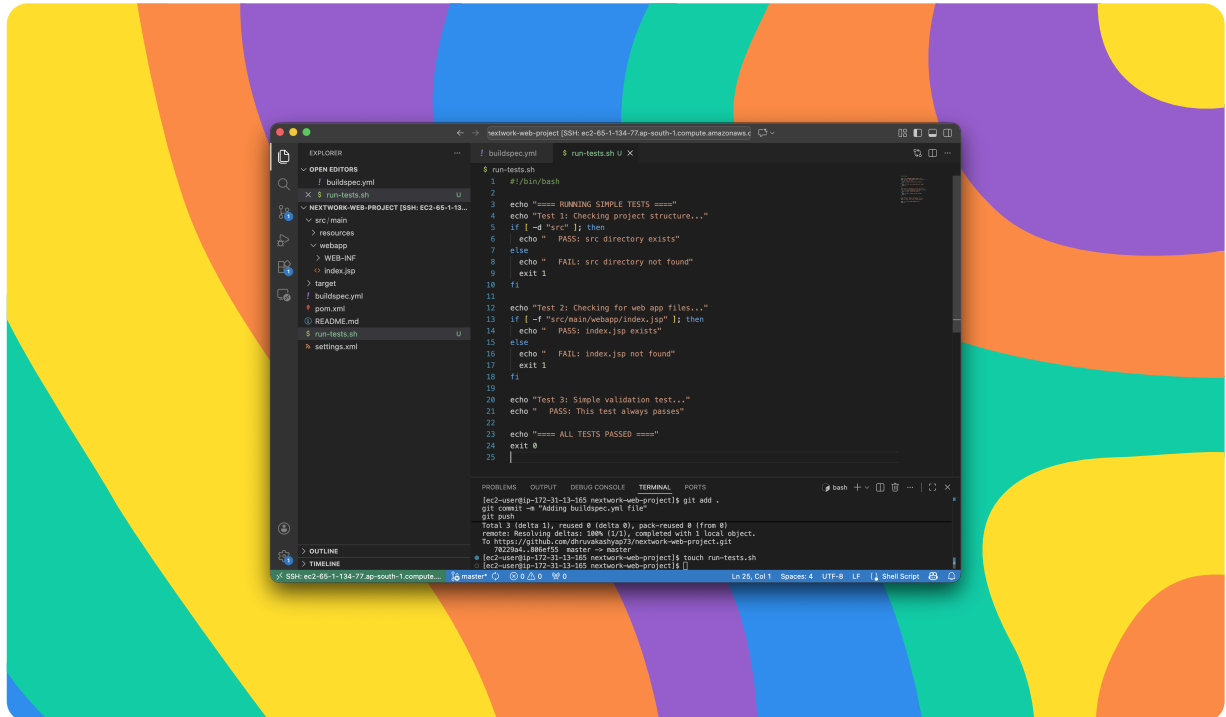


Automating Testing

The test script verifies that the project structure is correct by checking whether the src directory exists in the repository. It also confirms that the required web application file `src/main/webapp/index.jsp` is present before the build continues. These validations ensure the repository contains the expected files and fails the build early if critical components are missing.

The test script was added to the build process by updating the `buildspec.yml` file to run `run-tests.sh` before the Maven compile step. The script was made executable using `chmod +x` and executed during the build phase so the pipeline fails early if any validation test fails. After updating the `buildspec`, the changes were committed and pushed to GitHub so AWS CodeBuild could use the new automated testing workflow.

Automated testing was verified by reviewing the AWS CodeBuild build logs and confirming that the test phase markers appeared before the compilation steps. The log output showed that `run-tests.sh` was executed and returned successful "PASS" results, indicating the script ran as part of the pipeline. This confirmed that tests are now automatically triggered during every CodeBuild execution using the updated `buildspec.yml`.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

